⚡ Accelerate Test Execution with Adaptive Test and Ultra Edge

Reference: RA_451

Welcome to the Adaptive Test DAS Demo — a powerful illustration of how test programs can be made smarter, more flexible, and responsive to real-time conditions using UltraEdge technology.

Unlock Smarter Testing with DAS and UltraEdge

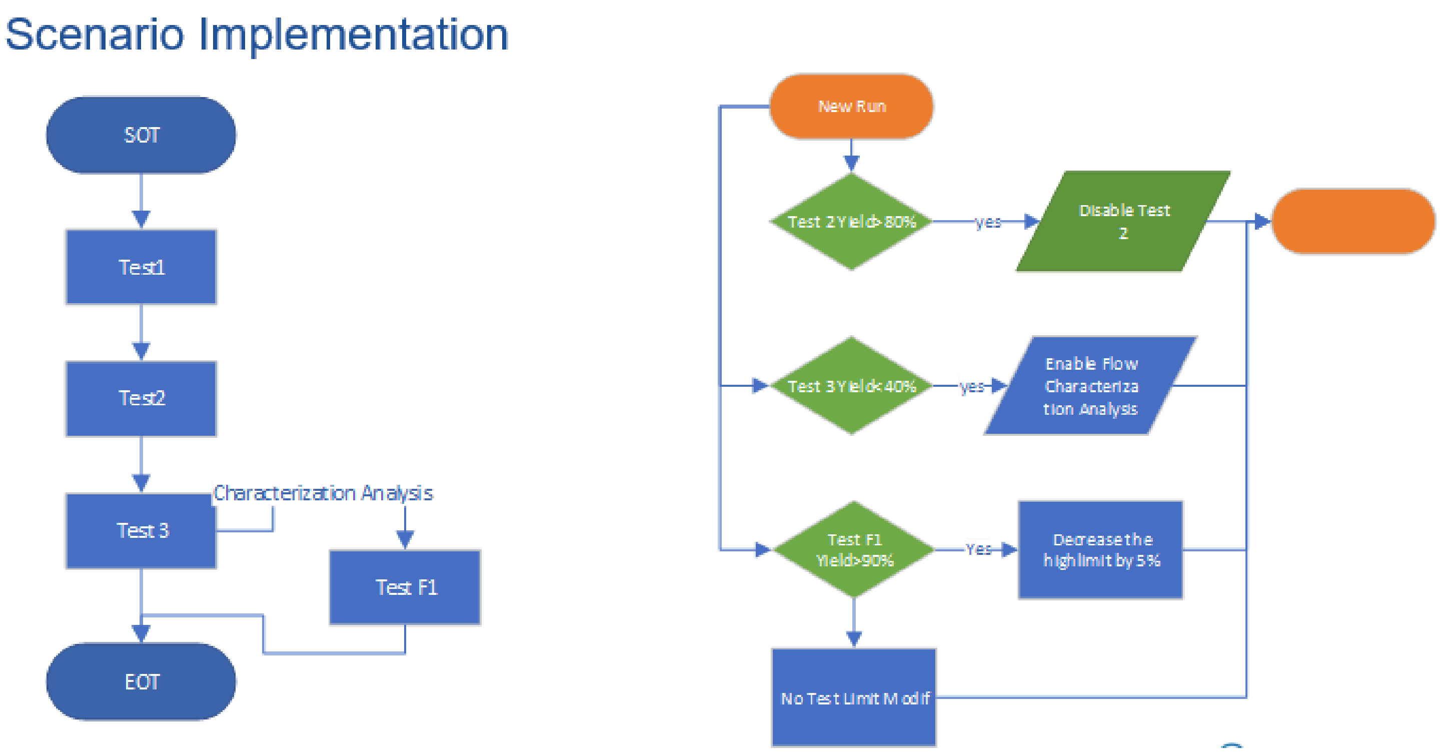
Welcome to the Adaptive Test DAS Demo — a powerful illustration of how test programs can be made smarter, more flexible, and responsive to real-time conditions. Whether you're running it locally or envisioning a scaled deployment on remote servers, this demo is built to showcase adaptability. As well during this demo we will use the UltraEdge computer in order to demonstrate how the powerfull computer can host your secured algorithm and interact with your test program without any modification of your test program.

What This Demo Demonstrates

Improving Test Efficiency with Archimedes In this scenario, we will address several challenges and demonstrate how Archimedes solutions can help resolve them. The system uses an algorithm that analyses current results to make decisions in real time:

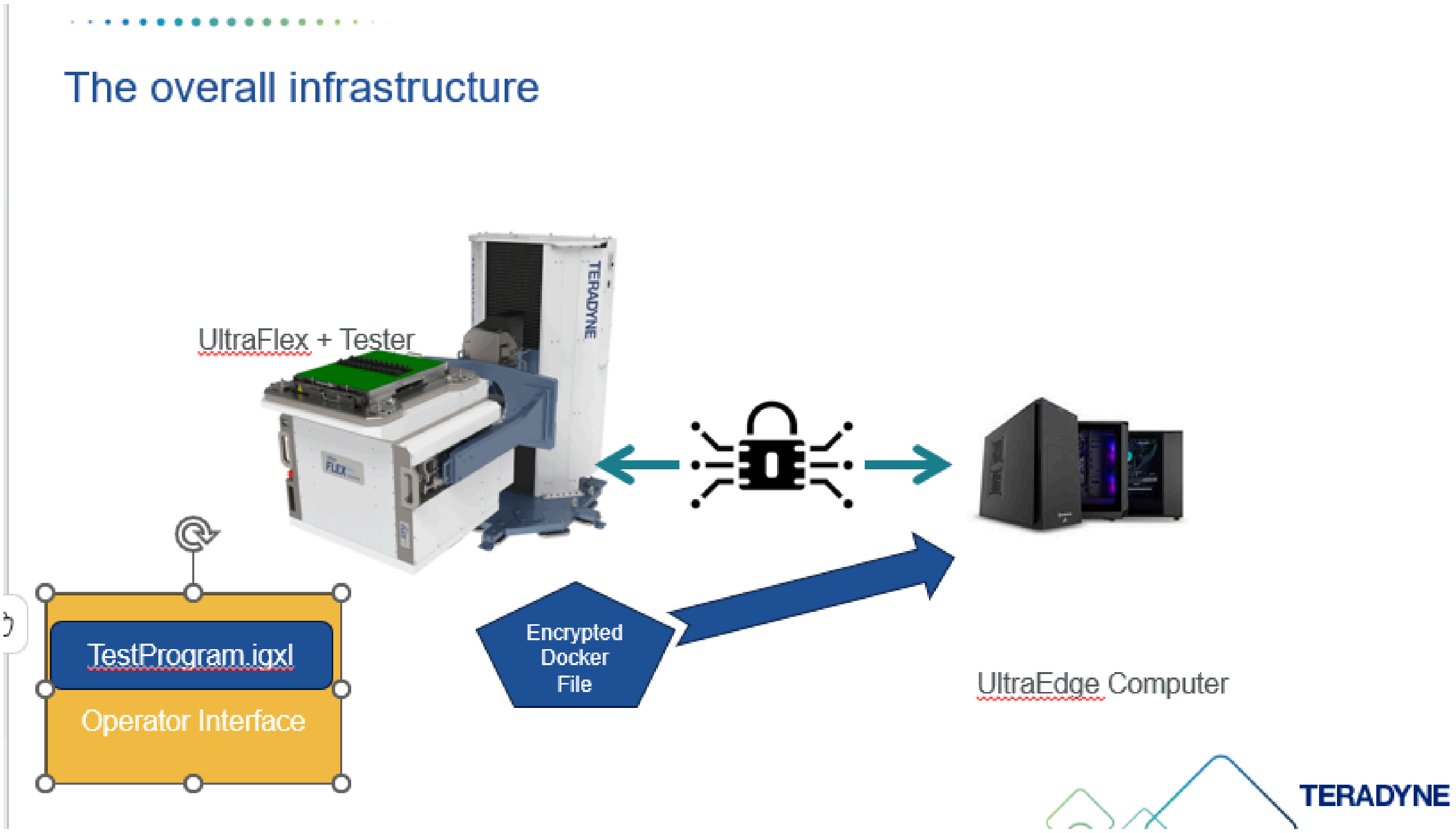
1. **Test Time Reduction**: If a test consistently achieves a certain yield threshold (e.g., 80%), the system will automatically disable that test to save time.
2. **Online Analysis**: When a test fails too frequently, a diagnostic flow will be automatically triggered to help identify and analyse the root cause.
3. **Quality Improvement**: Based on ongoing test results, the system will adapt test limits to better match the characteristics of the devices under test.

Which algorithm



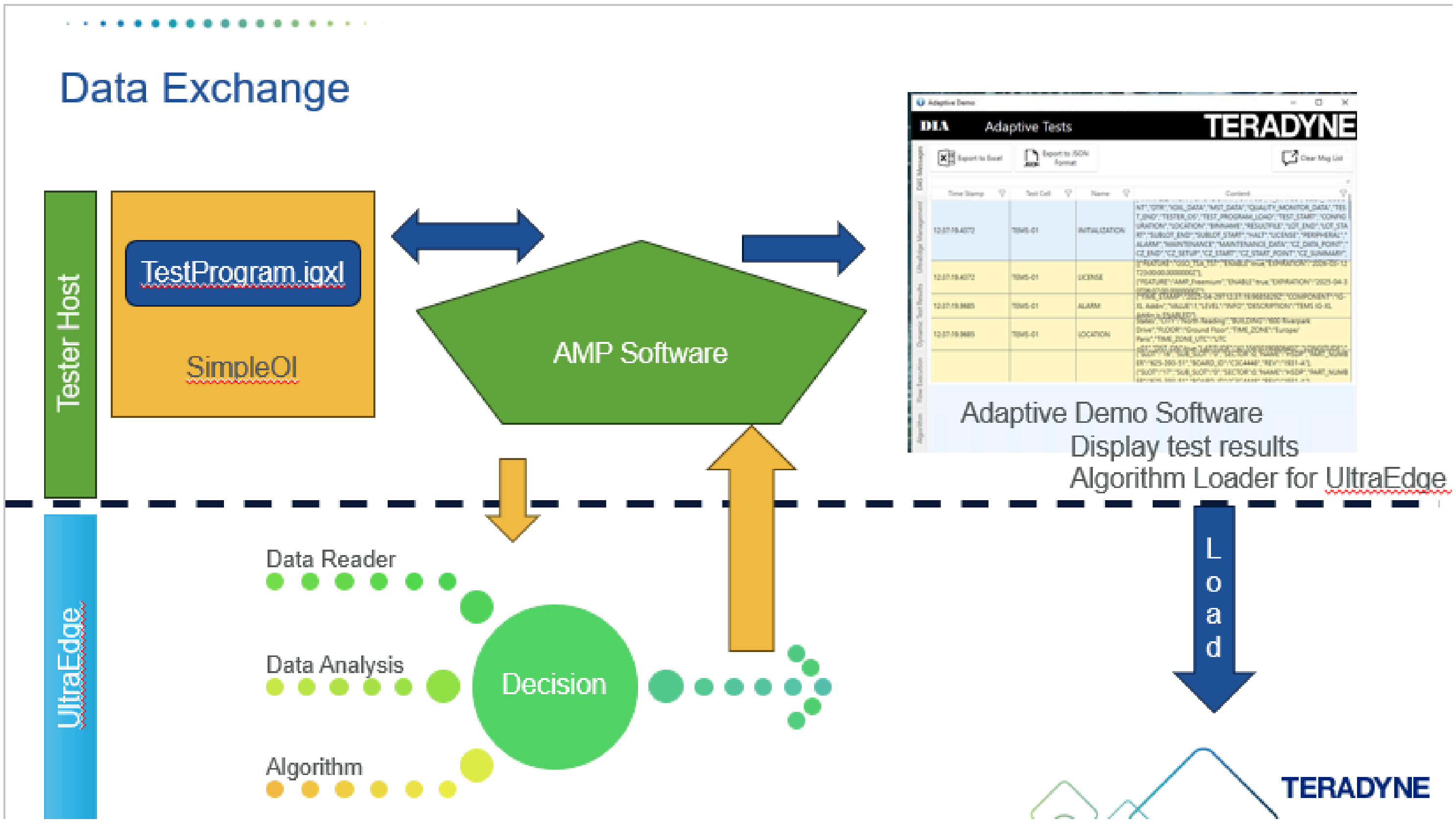
1. On the **left**, we see the structure of the test program: A simple sequence with three main tests—**Test1**, **Test2**, and **Test3**—and a sub-flow (disabled by default) that includes **Test F1**, which uses test limits for characterisation analysis.
2. On the **right**, we detail the logic that implements the scenario described in the previous slide:
3. **Test Time Reduction**: If **Test2** reaches a yield of **80% or more**, it will be **automatically disabled** to optimise test time.
4. **Online Analysis**: If **Test3**'s yield drops **below 40%**, the system **will activate the sub-flow** and execute **Test F1** for further characterisation.
5. **Quality Improvement**: If **Test F1** is active and its yield remains **below 90%**, the system will **automatically reduce the high limit** of the test by **5%**, continuing until the yield improves.

Overall Infrastructure



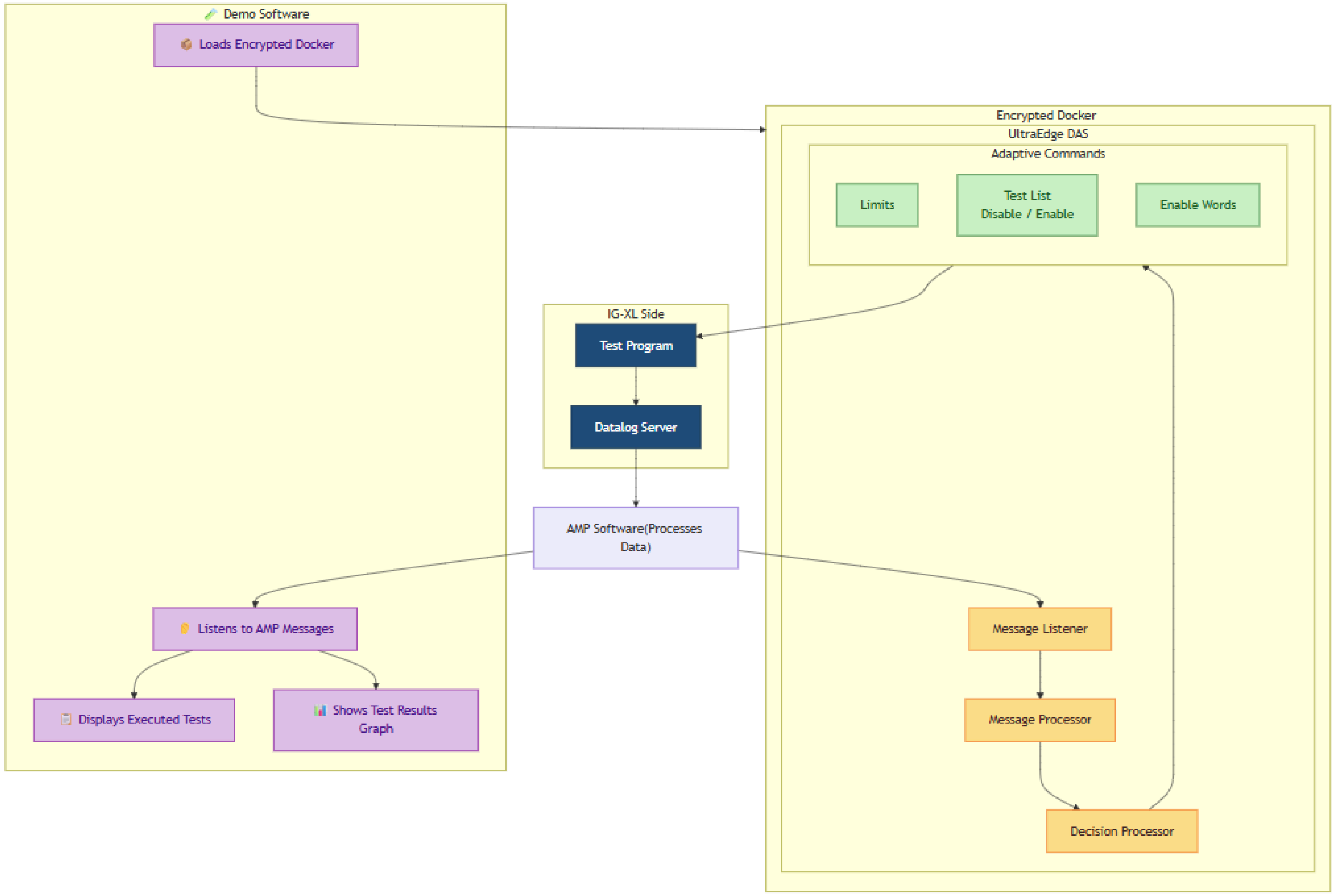
This image illustrates the infrastructure used to deploy and protect the algorithm implementation described earlier. The Python-based scenario is packaged inside an **encrypted Docker file**. This file is initially stored on the **tester host**, then securely **uploaded to the UltraEdge computer**. Once it reaches the UltraEdge system, the Docker is **decrypted locally** and executed. This secure mechanism ensures that your **intellectual property is fully protected**. Even if you are testing your product in a third-party Test House, your algorithm—potentially containing sensitive logic or proprietary data—remains secure and inaccessible from outside. Importantly, **the test program itself remains unchanged**. Thanks to AMP, it is possible to interact with it directly and securely, without modifying the original test flow.

Data Exchange



This images illustrates how data exchange is managed between the components. For demonstration purposes, we have implemented a system that captures and displays test data in real time. **Note that this demo system does not interact with the test program itself**—its role is only to display data, making the demonstration more engaging and easier to follow. Additionally, this system is responsible for copying the **algorithm from the encrypted Docker file** to the **UltraEdge (UE) computer.** Once the copy is complete, it acts as an **observer**, helping us showcase the type of data that can be captured via the **AMP software**. Once the algorithm is decrypted on the UltraEdge computer, it begins **reading the data received from the AMP software**. It then analyses this data using the algorithm and takes a **decision** accordingly. The resulting actions are then automatically applied to the **test program**. It's important to highlight that **no specific code needs to be embedded in the test program** to support these adaptive decisions. The interaction is handled entirely through AMP and the UltraEdge system.

Software Overview

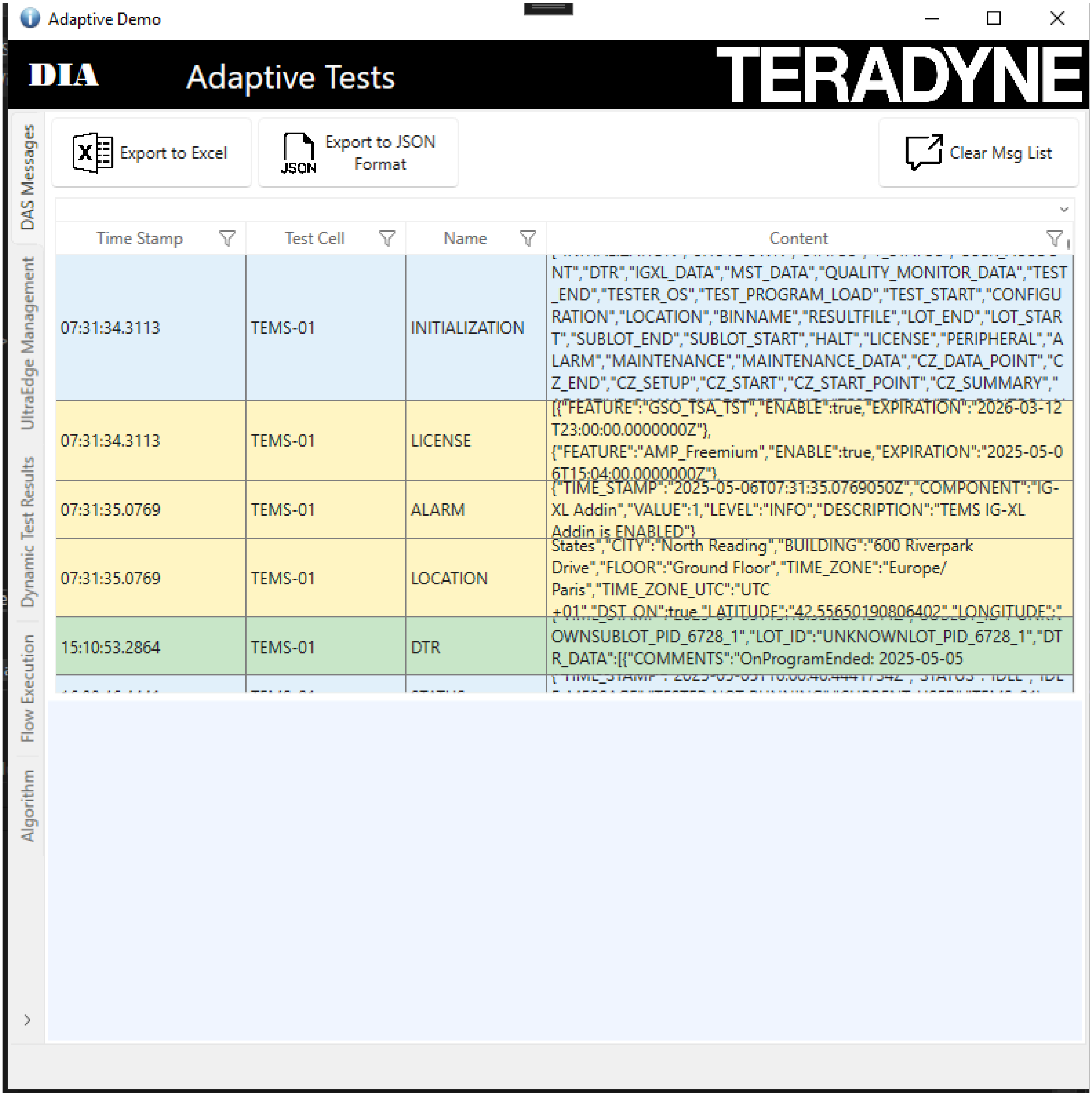


Explore the Demo Software

The demo application includes four dedicated sections:

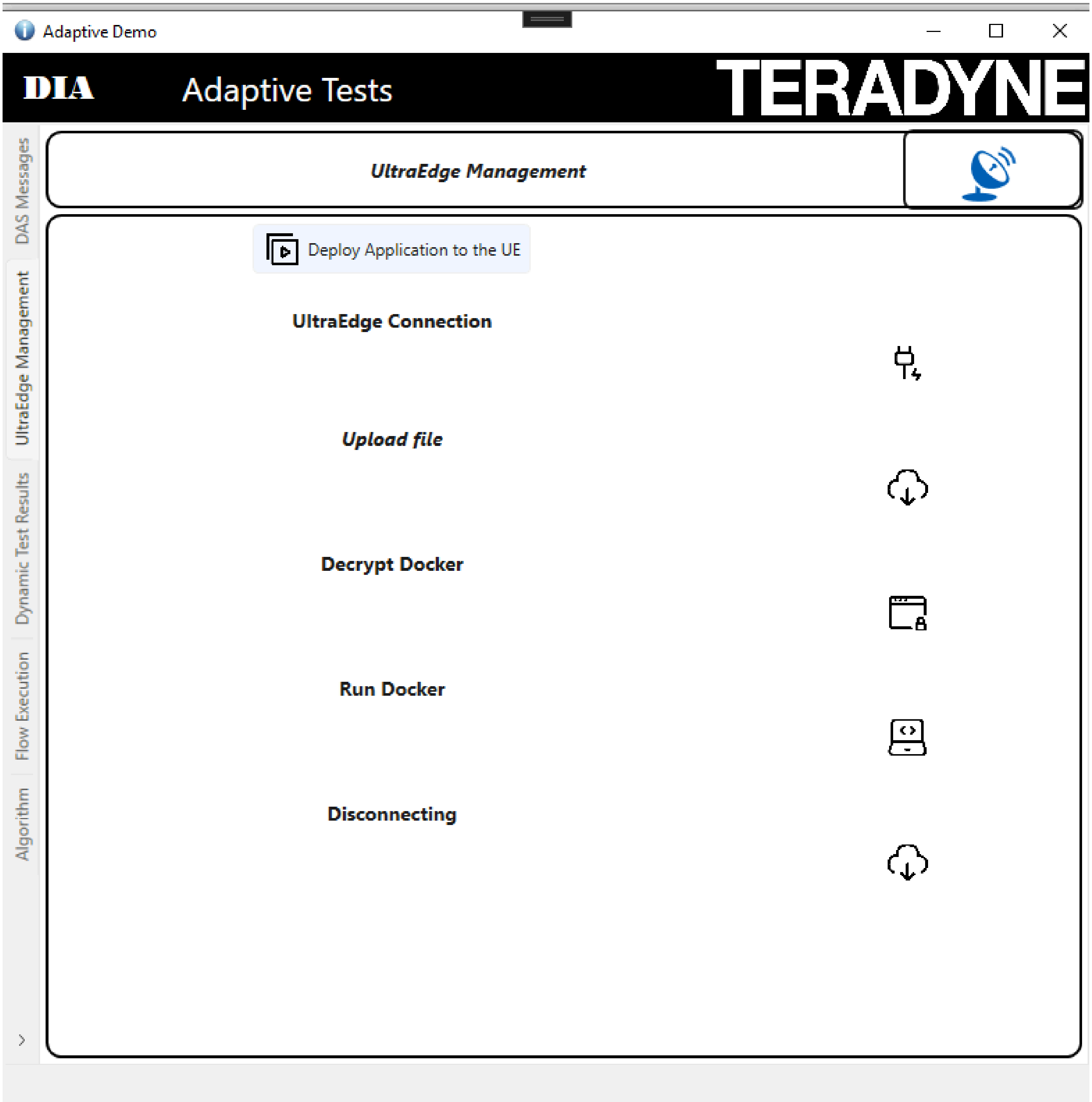
DAS Section

View all messages generated by the tester. Clicking on a message reveals its detailed contents.



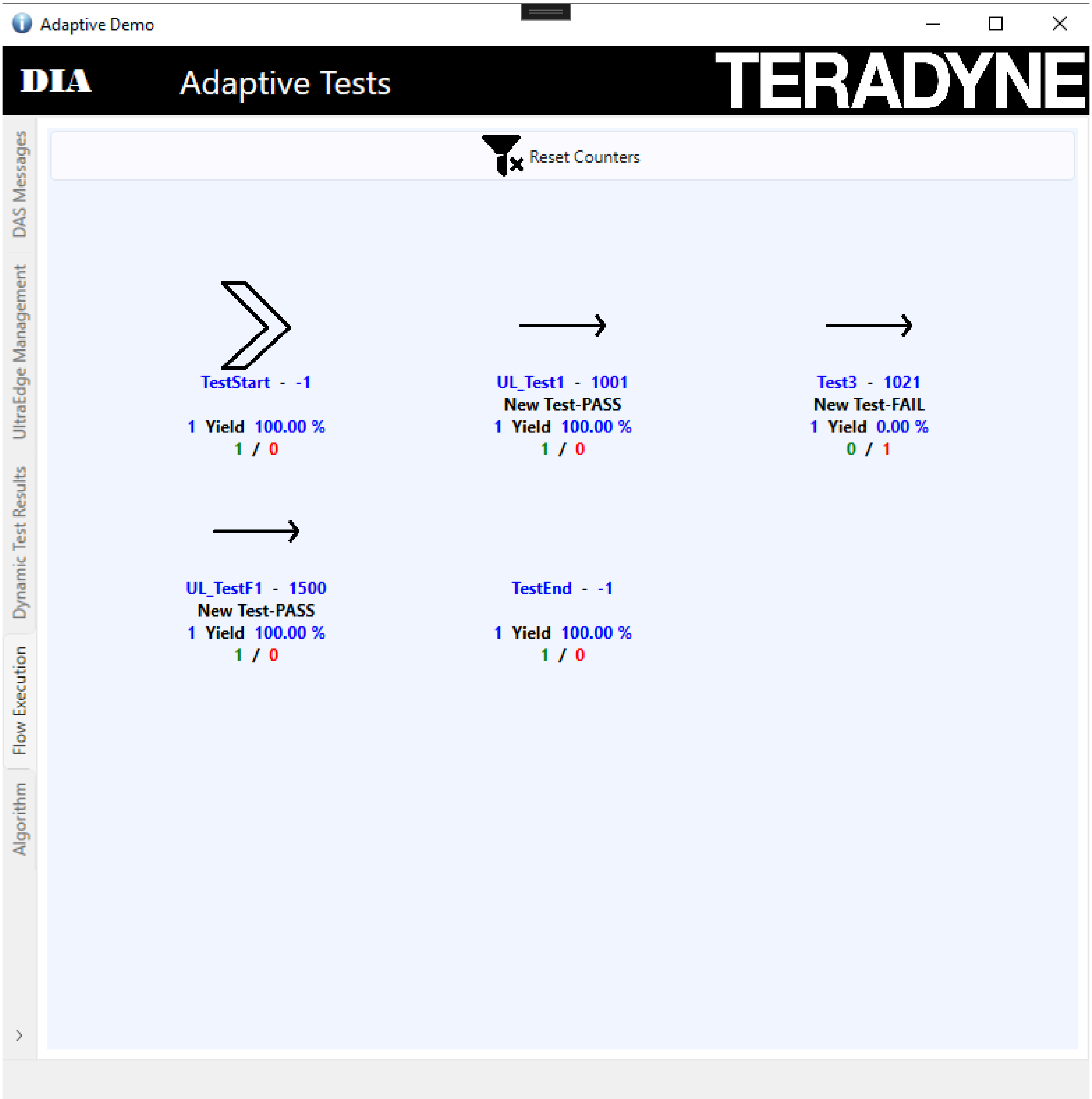
UltraEdge Section

Into this tab you will be allowed to upload the docker file into the **UltraEdge System**



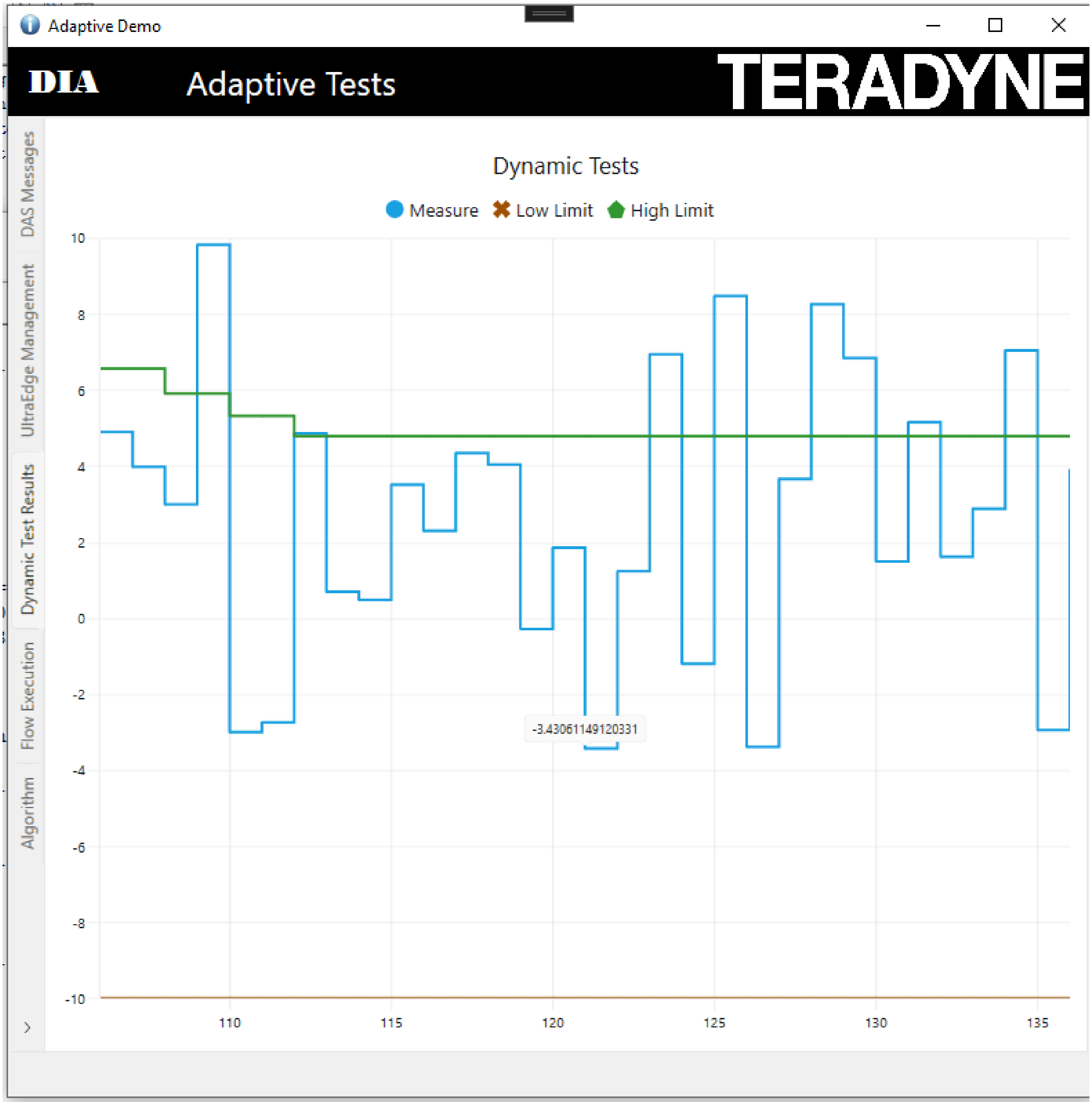
Flow Section

Into this section the system will show you which test has been executed and as well if a test has been disabled through an Adaptive Command.



Test Results Display Section

Displays the test results for `Test F1` and its associated limits.



Technical Snapshot

Component	Version
Language	C#
Framework	.NET 8

Component	Version
Environment	Windows 10
Excel	2016 (64-bit)
IGXL	10.60.02

Extend the Demo to Your Environment

To implement a similar system, we recommend reviewing the following guides:

Reference Architecture	Description
RA_468	How to implement a DAS in C#
RA_465	How to send an adaptive command in Python

See It in Action

Explore the complete Instructions to Run the Demo or browse the image gallery to discover key functionalities Looking to integrate this approach into your test strategy? Contact our team to explore how DAS can support your roadmap.

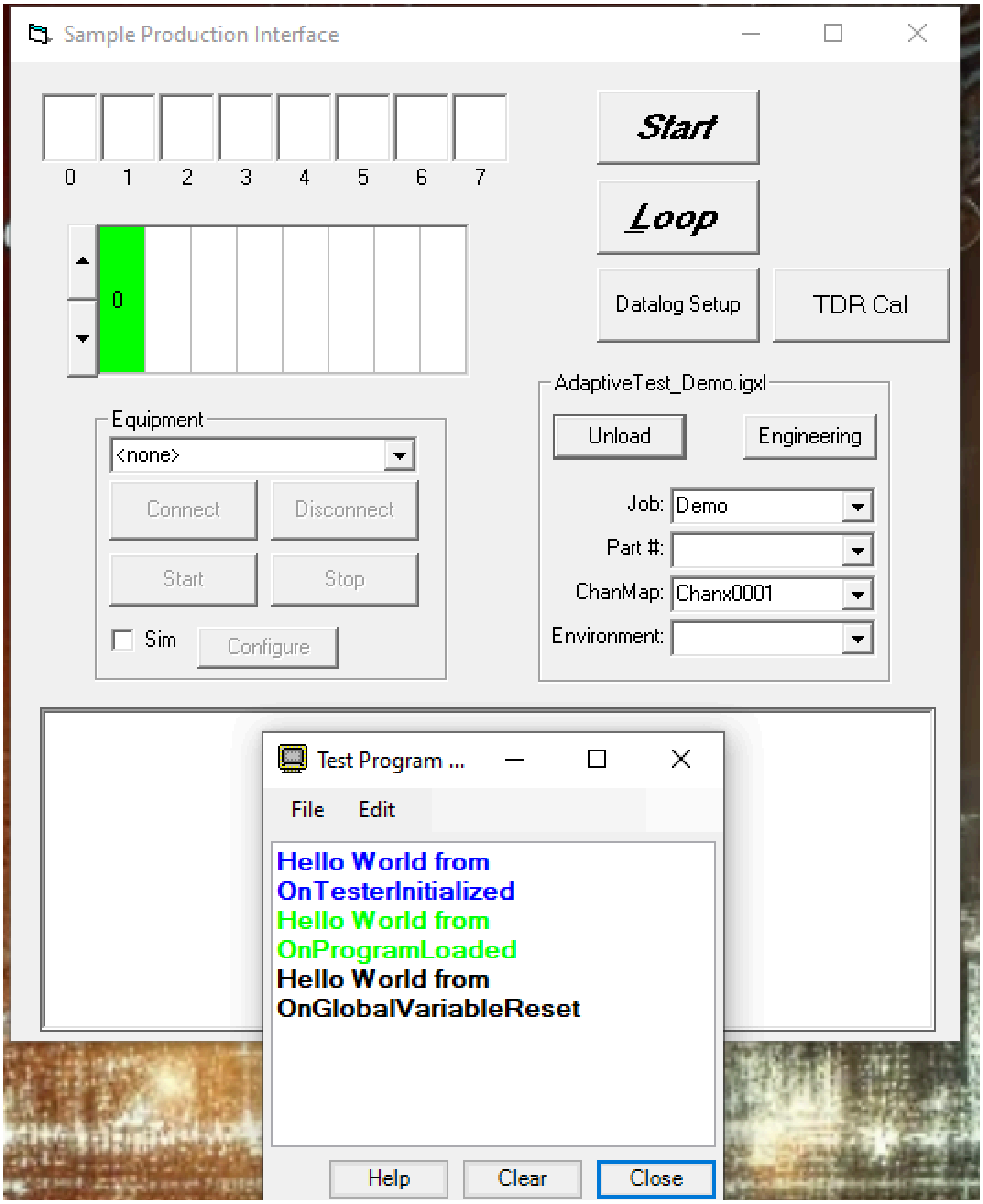
How to Run the Adaptive Test Demo with an UltraEdge

Reference: RA_451

This section provides step-by-step instructions to explore the key features of the Adaptive Test DAS Demo by using an UltraEdge.

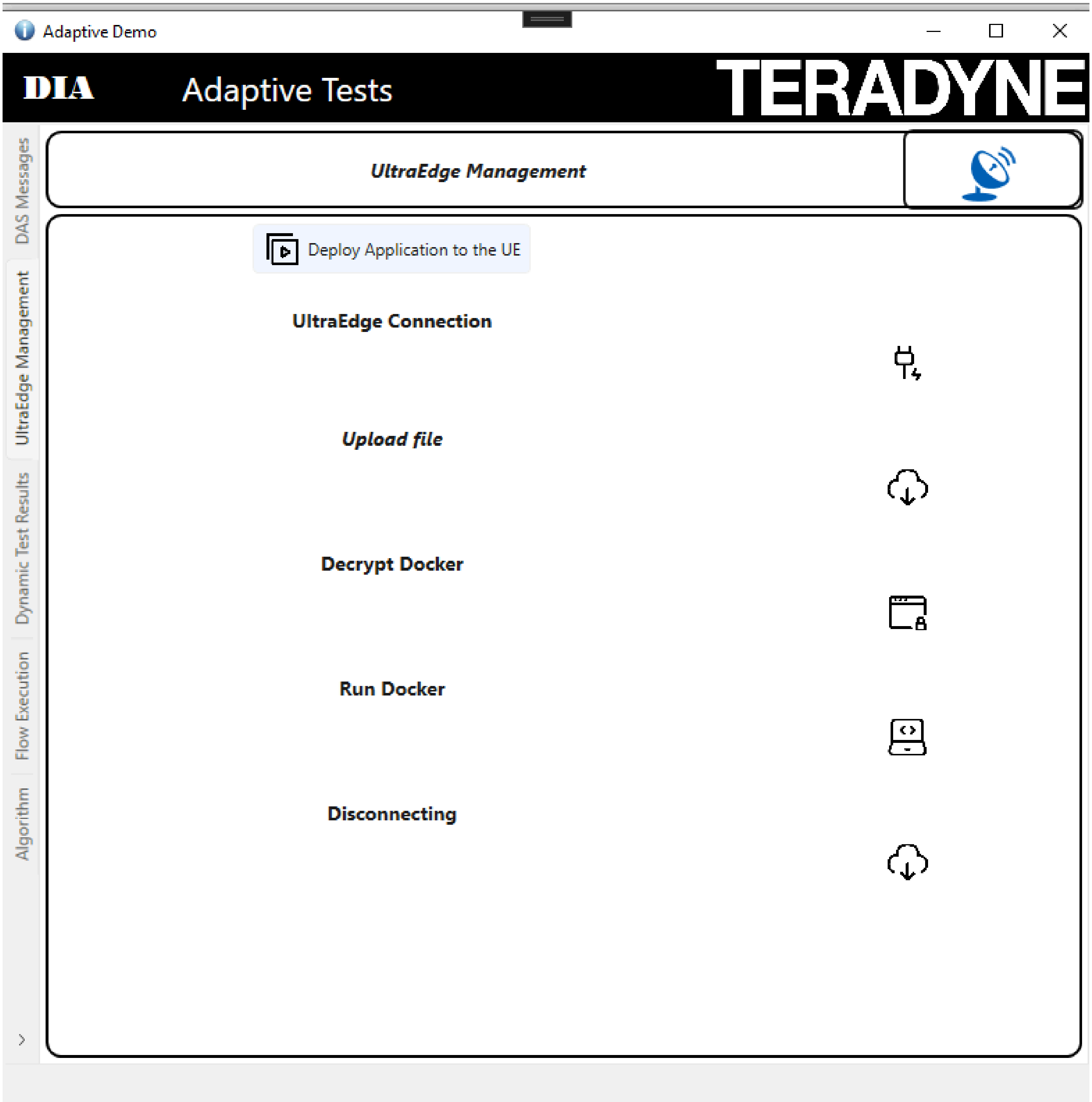
Step 1: Load the Test Program

Use the SimpleOI interface to load your test program.



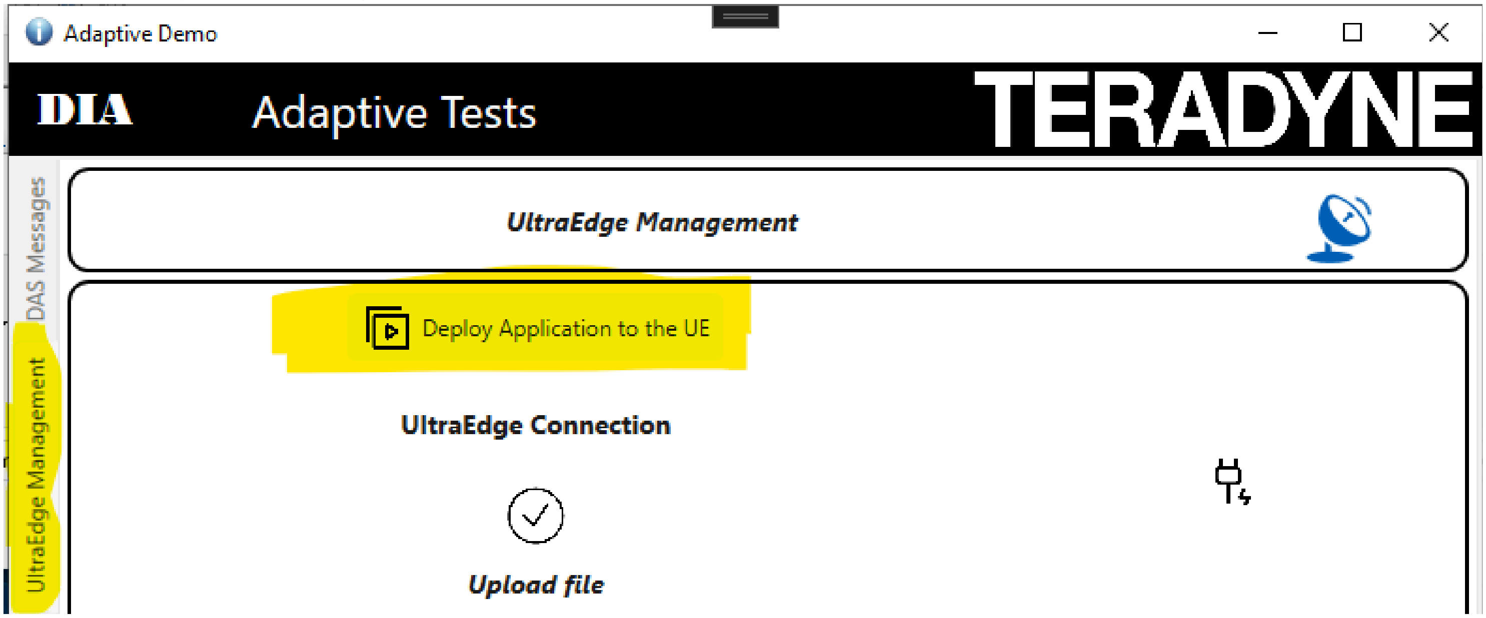
Step 2: Start the DAS

Start the DAS in order to interact with the UltraEdge System



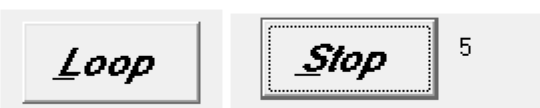
Step 3: Start the DAS

Be sure that your UltraEdge VM is started. Click on "*Deploy Application to the UE*"



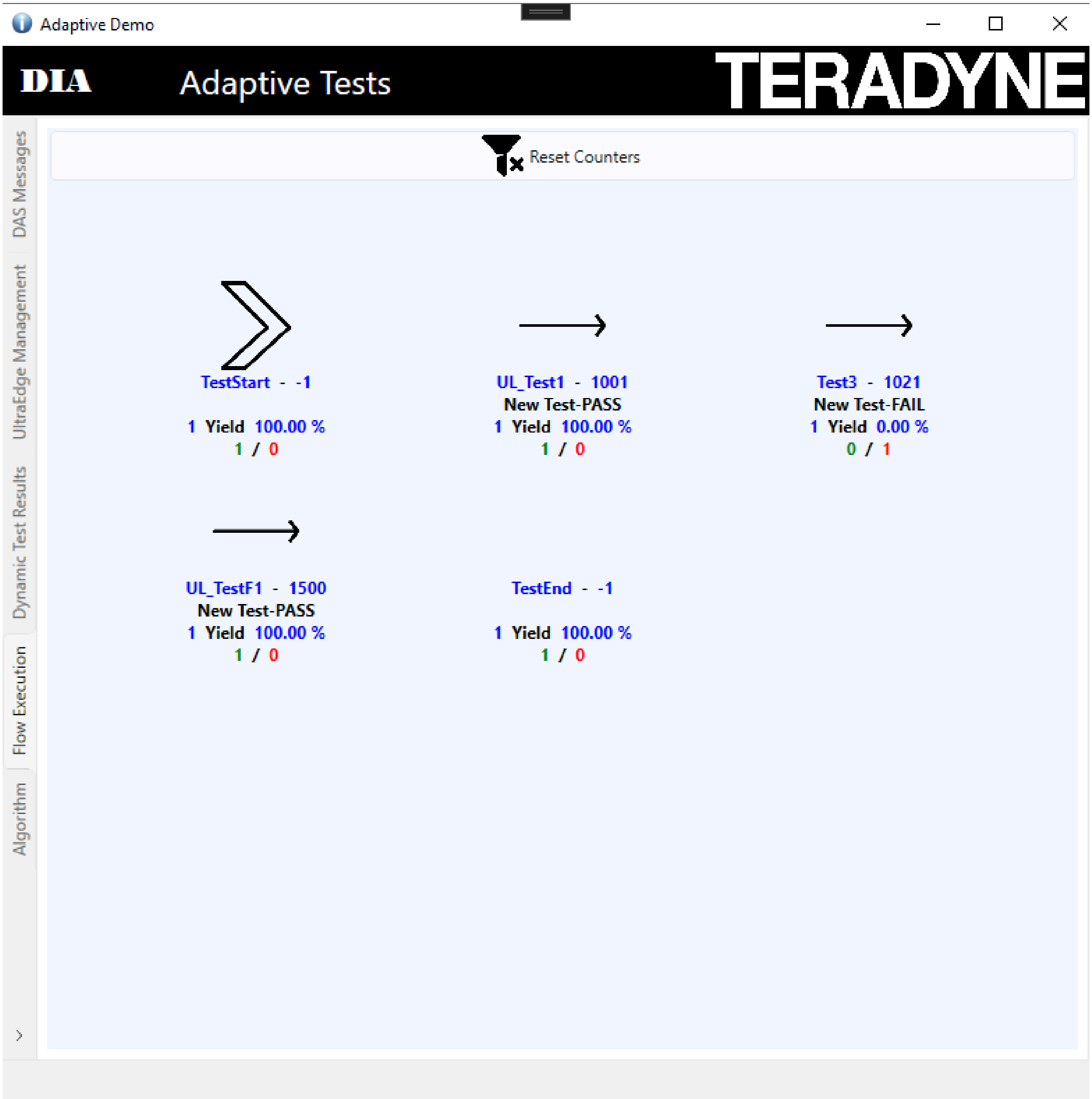
Step 4: Execute the Program in Loop Mode

Click the **Loop** button to execute the program 10 times. Once completed, press the **Stop** button.

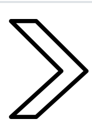


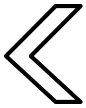
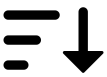
Step 5: Visualize the Test executed

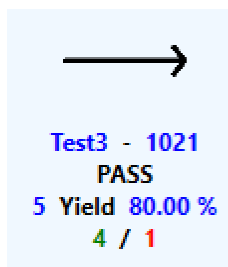
In the demo application, click **Update Test Limits**. Use the slider to modify the thresholds, then click **Send Limits** to apply them.



The following table represents the different tables

Icon	Description
	Test Start Icon

Icon	Description
	Test End Icon
	Test Executed
	Test Not Executed
	Test Disabled by an Adaptive command
	Limits updated by an Adaptive command

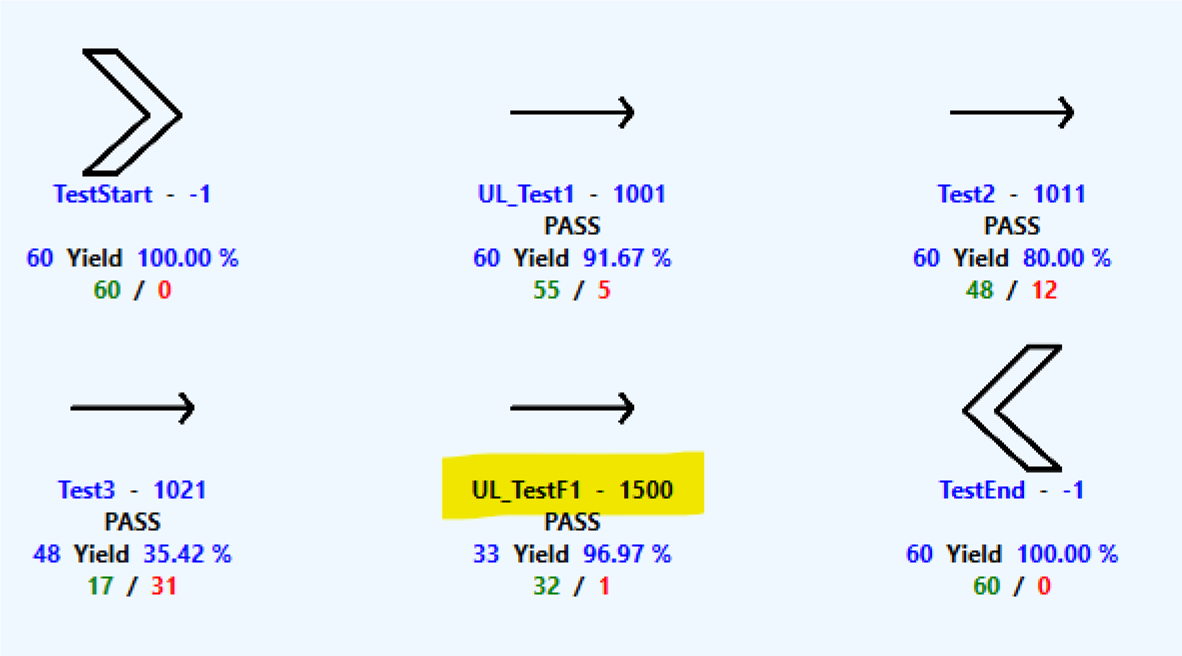
For each test the following data will be displayed 

| Data | Description | |-------------|-------------------| | Test3 | Represents the test name | | 1021 | Represents the test number | | PASS | The last execution was PASS | | 5 | Number of times the test has been executed | | 80.00% | Yield of this test | | 4 | Number of Pass | | 1 | Number of Failure |

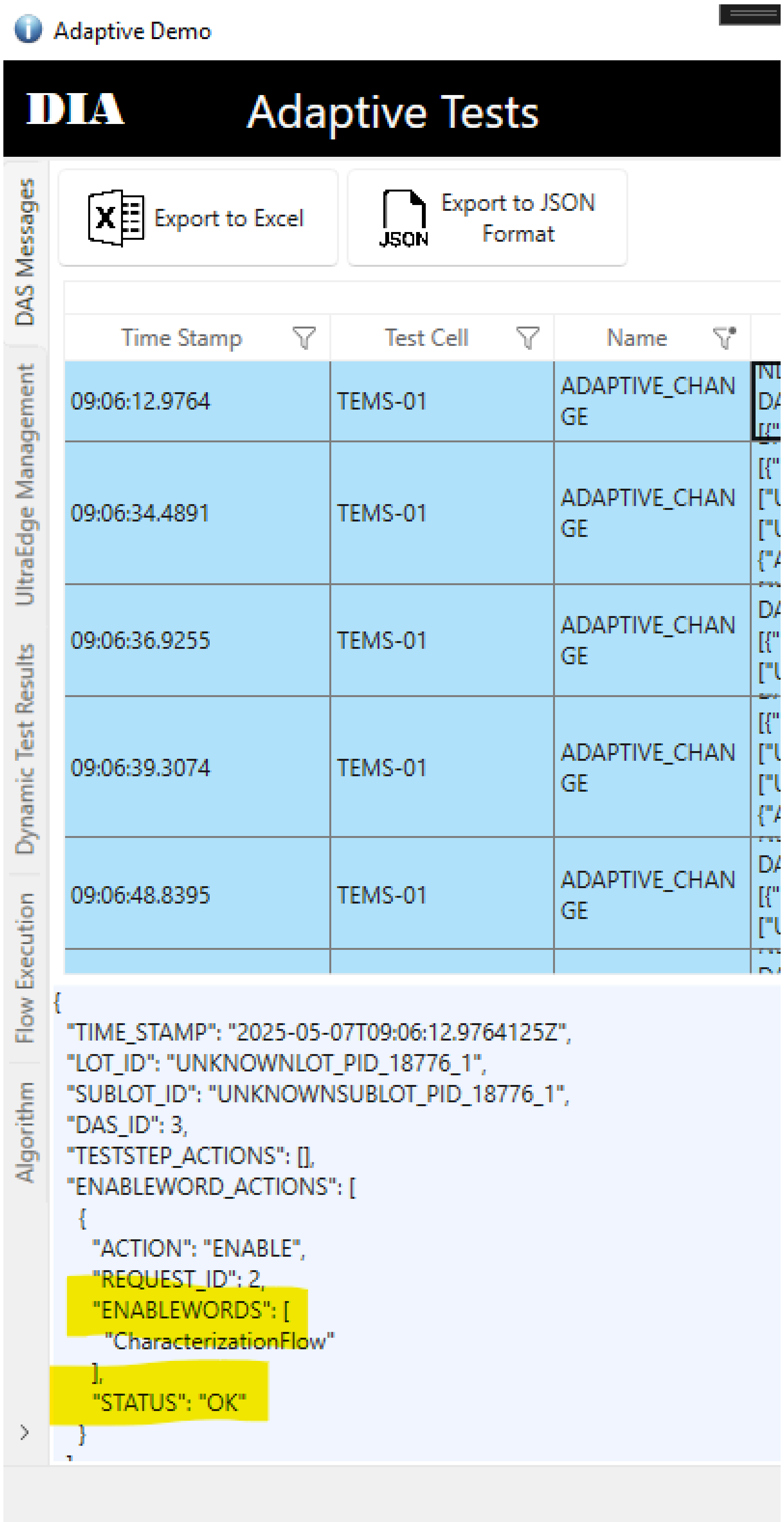
Run the test a few times from the SimpleOI interface to observe the impact.

Step 6: Observe the New test Activated

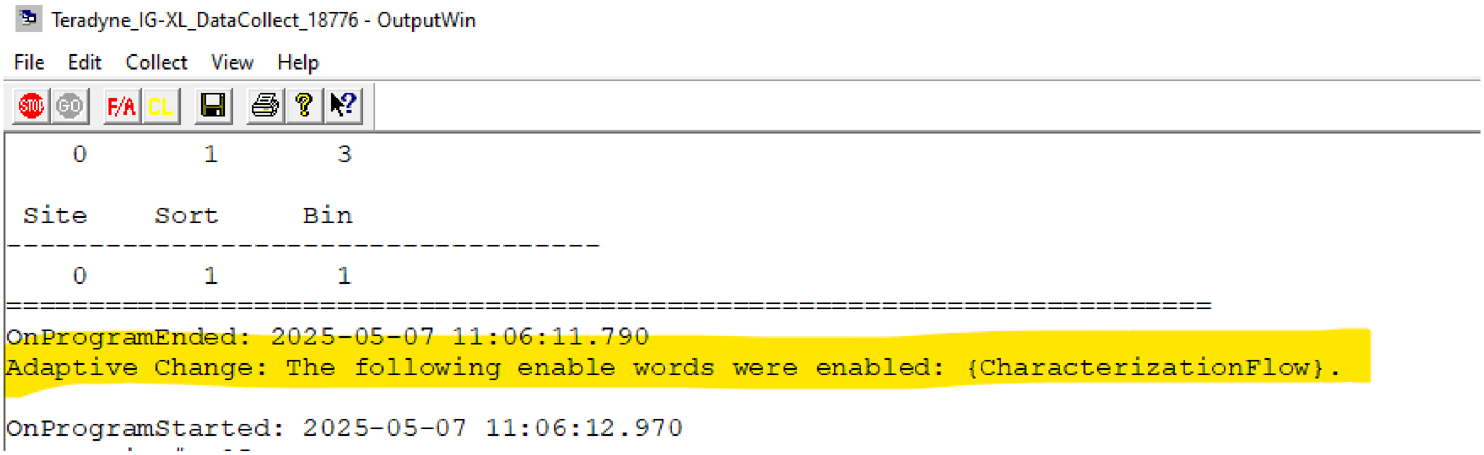
Loop the test program till you observe that the new test has been enabled. After few runs you will see that the test UL_TestF1 has been activated.



As well you can observe that a new message has been generated

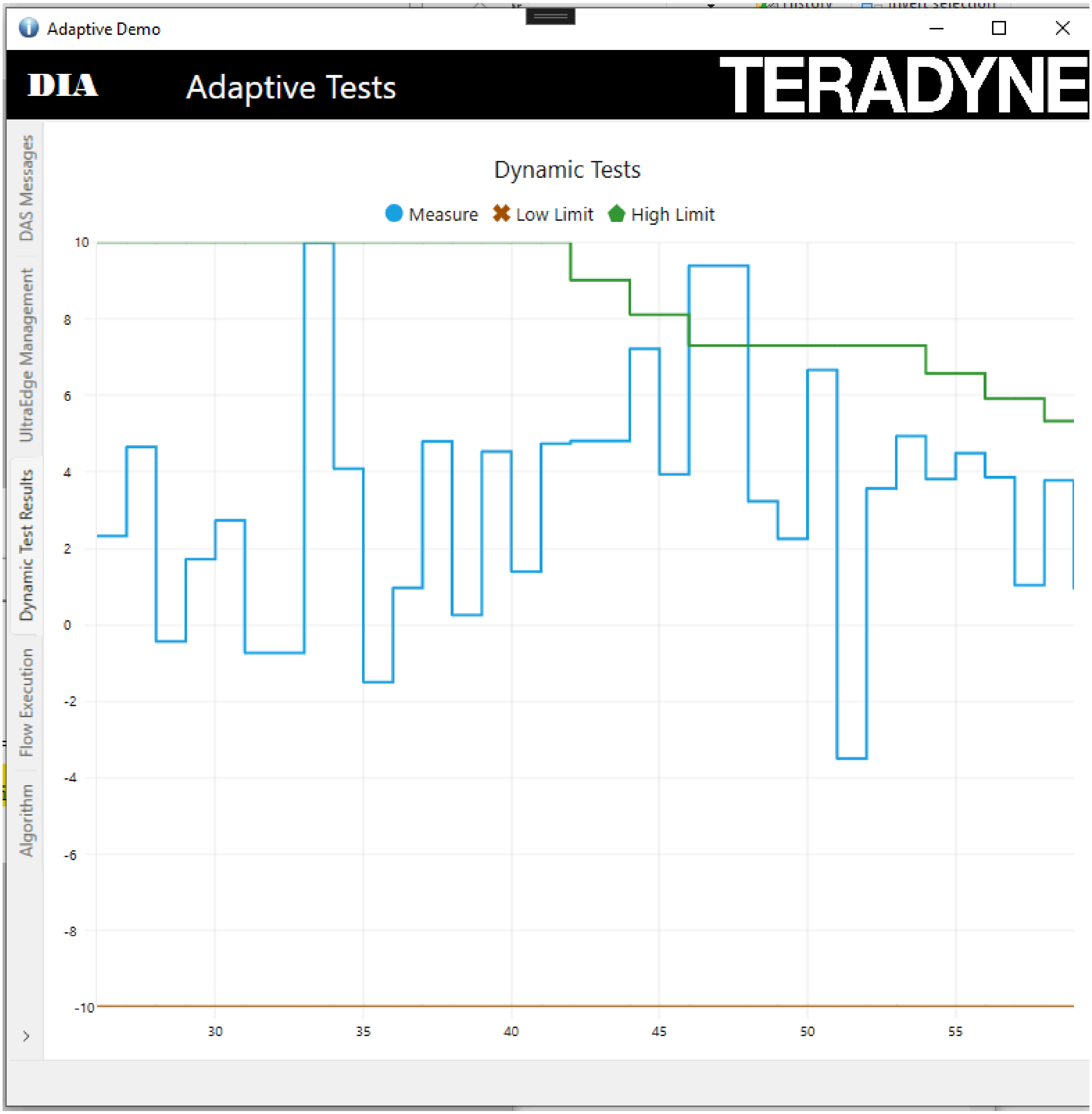


And as well a message will be added into the test program output



Step 7: Automatic Test Limit Updated

After few tests, you will observe that the high limit of the TestF1 will be updated. You can observe that into the tab *Dynamic Test Results*



And you can observe some messages into the test program output

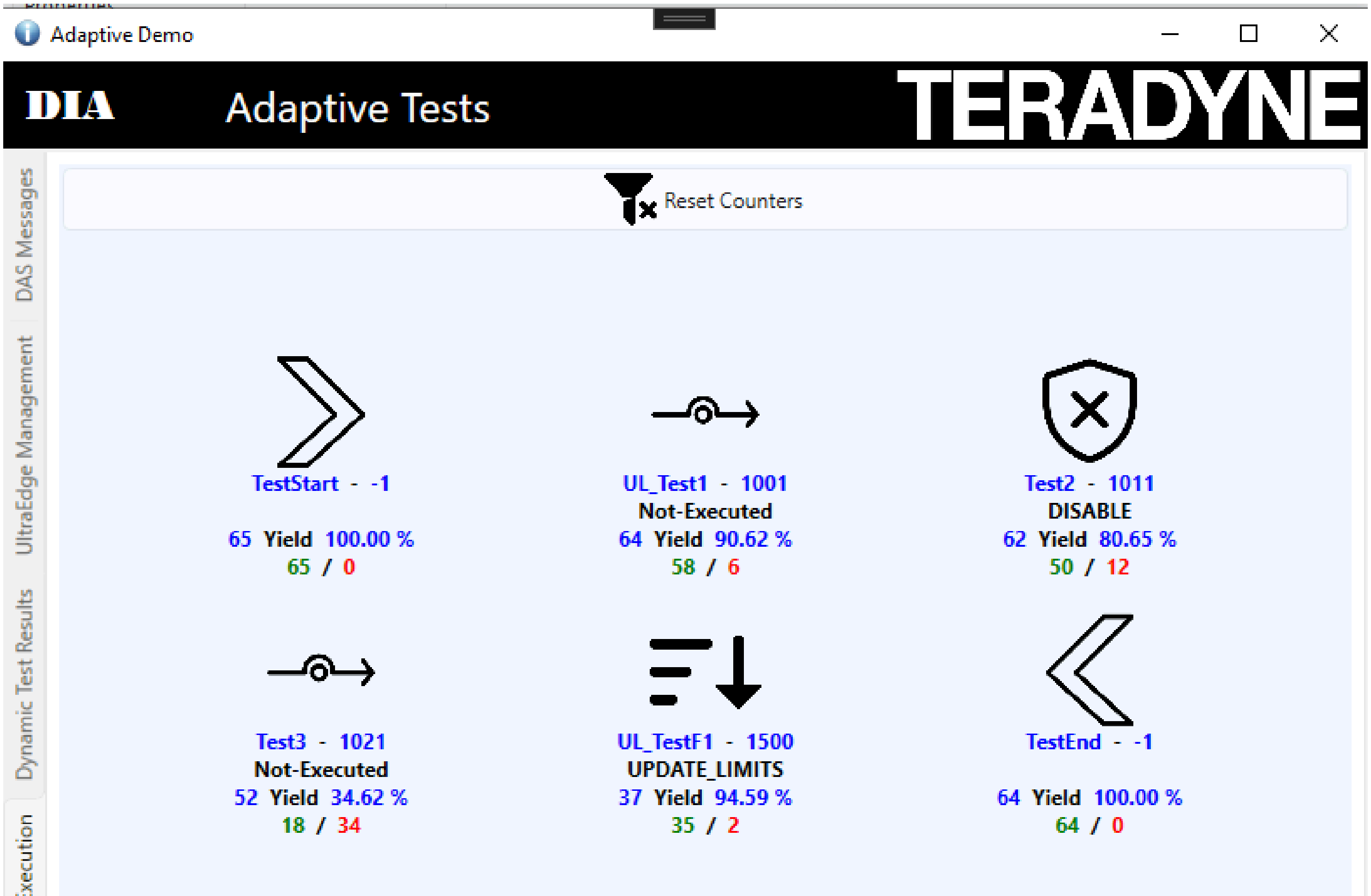
```
OnProgramEnded: 2025-05-07 11:06:35.780
Adaptive Change: The limits {UL_TestF1} on test step {UL_TestF1} were changed. New low limit = -10. New high limit = 8.1.

OnProgramStarted: 2025-05-07 11:06:36.930
    Device#: 45

 Number   Site  Test Name      Pin  Channel  Low            Measured           High          Force          Loc
<Icc_Fake>
 1001      0    UL_Test1             -1      -5000.0000 mA  -1041.5572 mA      4000.0000 mA  0.0000         0
Run executed 45
<Test2>
 1011      0    Test2                -1      0.0000 mA      693.6616 mA        5000.0000 mA  0.0000         0
Run executed 34
<Test3>
 1021      0    Test3                -1      0.0000 mA      7787.5051 mA   (F) 5000.0000 mA  0.0000         0
OnePin
Run executed 19
<OnePin_UserLimit_VBT>
 1500      0    UL_TestF1            -1      -10000.0000 mA 7205.5733 mA       8100.0000 mA  0.0000         0
```

Finally we can observe Adaptive Message are generated


```
"LOT_ID": "UNKNOWNLOT_PID_18776_1",
"SUBLOT_ID": "UNKNOWNSUBLOT_PID_18776_1",
"DAS_ID": 3,
"TESTSTEP_ACTIONS": [
 {
  "ACTION": "UPDATE_LIMITS",
  "REQUEST_ID": 5,
  "TESTSTEP_NAMES": [
   "UL_TestF1"
  ],
  "LIMITS": {
   "LIMIT_NAMES": [
    "UL_TestF1"
   ],
   "LO_LIMIT": -10,
   "HI_LIMIT": 8.1
  },
  "STATUS": "OK"
 }
],
"ENABLEWORD_ACTIONS": []
```

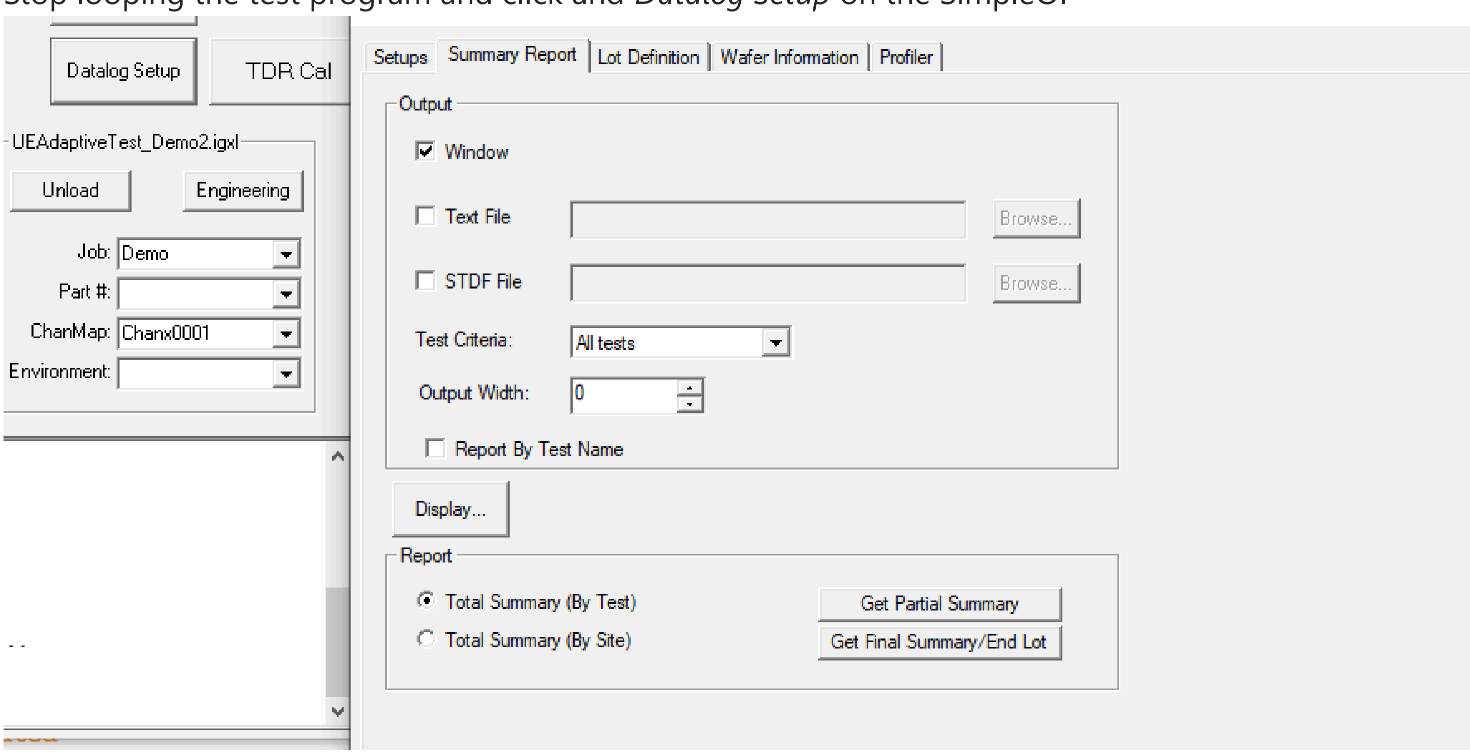

Step 8: Test Disabled

As soon as the test2 reachs the yield of 80% it will be disabled by an Adaptive test command

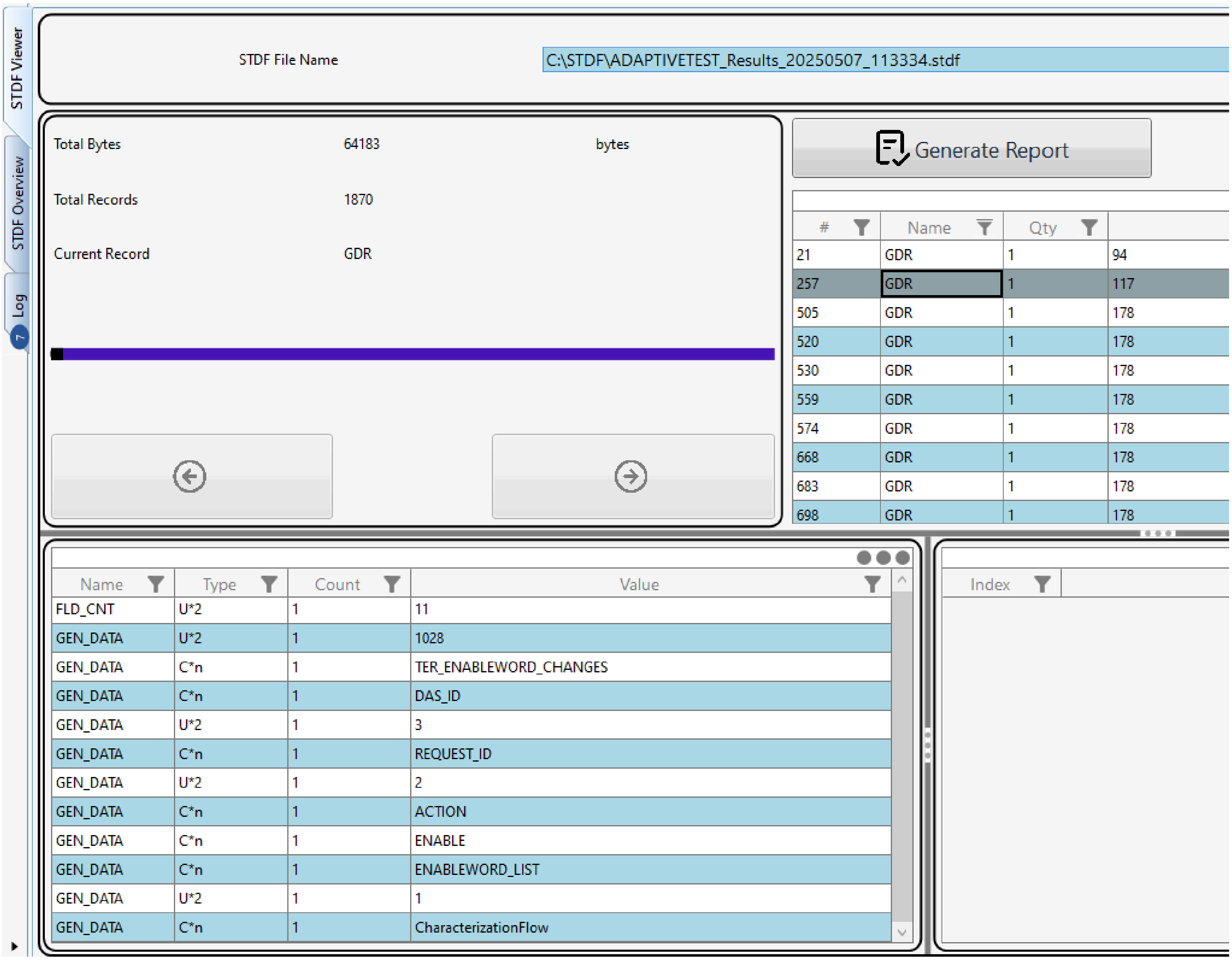


Step 9: Check the STDF File

Stop looping the test program and click and *Datalog Setup* on the SimpleOI



Click on Get Final Summary End lot With an STDF Browser check the most recent STDF file located into *C:\STDF*



Download the Demo

[Download the demo from here](#)